# CSC317 Tutorial 8

Mass-Spring Systems

In this assignment, you will implement

$$\{\mathbf{p}^t, \mathbf{p}^{t-\Delta t}\} \rightarrow \mathbf{p}^{t+\Delta t}$$

such that the sequence of positions obey the physics of mass-spring systems.

Given current and previous timestep positions, $p^{t+\Delta t}$ can be computed as the minimizer of a non-linear optimization problem.

$$\mathbf{p}^{t+\Delta t} = \arg\min_{\mathbf{P}} \sum_{ij} \frac{1}{2} k \left( \min_{||\mathbf{d}_{ij}||=r_{ij}} \|(\mathbf{p}_i - \mathbf{p}_j) - \mathbf{d}_{ij}\|^2 \right)$$
$$+ \frac{\Delta t^2}{2} \left( \sum_i m_i \left( \frac{\mathbf{p}_i - 2\mathbf{p}_i^t + \mathbf{p}_i^{t-\Delta t}}{\Delta t^2} \right)^2 \right) - \left( \sum_i \mathbf{p}_i^\top \mathbf{f}_i^{\text{ext}} \right)$$
$$+ \text{penalty term for pinned vertices}$$

To solve it, we saw the following local-global algorithm in lecture:

1. Local step: Given current values of $\mathbf{p}$ determine $\mathbf{d}_{ij}$ for each spring.
2. Global step: Given all $\mathbf{d}_{ij}$ vectors, find positions $\mathbf{p}$ that minimize the quadratic energy.
3. If not satisfied, go to Step 1.

To make life easier, let's define the following matrices:

- $E \in \mathbb{R}^{(m \times 2)}$ is a matrix of edges storing vertex indices.
- $A \in \mathbb{R}^{m \times n}$ is a matrix storing the signed incidences:

$$A_{e,k} = \begin{cases} 1 & \text{if } k \text{ is the first vertex of } e \\ -1 & \text{if } k \text{ is the second vertex of } e \\ 0 & \text{otherwise} \end{cases}$$

- $M \in \mathbb{R}^{n \times n}$ is the diagonal matrix of masses.
- $C \in \mathbb{R}^{p \times n}$ is the selection matrix for the pinned vertices, with $p$ pinned vertices. That is, $C_{i,j} = 1$ if the $i$th pinned vertex has index $j$, and 0 otherwise.

The energy we saw was super complicated. Some more definitions to help us:

$$Q_1 := kA^TA + \frac{1}{\Delta t^2}M \in \mathbb{R}^{n\times n}$$

$$Q_2 := \frac{\partial}{\partial p}\left(\frac{1}{2}wp^T(wC^TC)p\right) \in \mathbb{R}^{n\times n} \text{ (work this out!)}$$

$$Q := Q_1 + Q_2 \in \mathbb{R}^{n\times n}$$

$$y_1 := \frac{1}{\Delta t^2}M(2p^t - p^{t-\Delta t}) + f^{ext} \in \mathbb{R}^{n\times 3}$$

$$y_2 := \frac{\partial}{\partial p}\left(p^T(wC^TC)p_{rest}\right) \in \mathbb{R}^{n\times 3} \text{ (work this out!)}$$

$$b := kA^T\mathbf{d} + y_1 + y_2 \in \mathbb{R}^{n\times 3}$$

With these definitions, our algorithm becomes:

1. Local step: Given current values of $\mathbf{p}$, determine $\mathbf{d}_{ij}$ for each spring.
2. Global step: Given all $\mathbf{d}_{ij}$ vectors, find positions $\mathbf{p}$ that minimize the quadratic energy

$$\frac{1}{2}\mathrm{tr}(\mathbf{p}^T Q\mathbf{p}) - \mathrm{tr}(\mathbf{p}^T\mathbf{b})$$

   by taking the derivative and setting to zero, that is, solving $Q\mathbf{p} = \mathbf{b}$. Remember, $\mathbf{b}$ depends on $\mathbf{d}_{ij}$!
3. If not satisfied, go to Step 1.

```
void signed_incidence_matrix_dense(
  const int n,
  const Eigen::MatrixXi & E,
  Eigen::SparseMatrix<double>  & A)
{
```

Compute **A**, the $\#edges \times \#vertices$ matrix storing the signed incidences:

$$A_{e,k} = \begin{cases} 1 & \text{if } k \text{ is the first vertex of } e \\ -1 & \text{if } k \text{ is the second vertex of } e \\ 0 & \text{otherwise} \end{cases}$$

`fast_mass_springs_precomputation_dense`

- You're given: rest positions **V**, edges **E**, list of masses **m**, stiffness $k$, list of pinned vertices **b**, and time step $\Delta t$.
- You need to produce the following dense matrices:
  1. **M**, the diagonal matrix of masses
  2. **r**, a vector of rest lengths
  3. **A**, the signed incidence matrix (use previous function)
  4. **C**, the selection matrix, $p \times n$ where $\mathbf{C}_{i,j} = 1$ if the $i$th pinned vertex has index $j$.
  5. The matrix **Q** we defined a few slides ago.

`fast_mass_springs_step_dense`

Compute 50 iterations of a local-global solve, in which each iteration computes $\mathbf{d}_{ij}$ for each spring, and then solves $Q\mathbf{p} = \mathbf{b}$.

Some pointers:

- Note the differences between `Uprev` (positions at $t - \Delta t$) and `Ucur` (positions at $t$), and `V` (rest positions).
- Do not create a new variable named `b`, we are already using it for the list of pinned vertices.
- Identify what quantities among **Q**, **b**, and **y** are not constant across iterations.

Then implement the same methods for sparse matrices:

```
signed_incidence_matrix_sparse
fast_mass_springs_precomputation_sparse
fast_mass_springs_step_sparse
```

Some pointers:

- All large dense matrices should be converted to sparse. Vectors stay dense.
- Use Eigen's `setFromTriplets` to construct the sparse matrices:
  - Use a `std::vector` of `Eigen::Triplet<double>` to store the triplets.
  - `ijv.emplace_back(i,j,v);` means adding entry with value *v* at row *i* and column *j*.
- *Do not* add zero entries.
- *Do not* create a dense matrix then convert to sparse.

Good luck!